

Seguridad de la Información

Práctica 7: Servidor Web Seguro con Certificados de Cliente

TITULACIÓN: Grado en Ingeniería Informática

AUTOR: Sergio Acosta Quintana

Fecha: 3 de mayo de 2026

Índice general

Índice de figuras	2
7. Servidor Web Seguro con Certificados de Cliente	3
7.1. Preparación de los Certificados (Asignatura y FNMT)	3
7.2. Creación del Fichero de CAs Confiables para Apache	4
7.3. Modificación de la Configuración de Apache	4
7.4. Instalación del Script CGI	6
7.5. Rearranque de Apache y Verificación	6
7.6. Conexión de prueba con <code>openssl s_client</code>	7
7.7. Pruebas Finales en Firefox	9
7.7.1. Importación de Certificados	9
7.7.2. Conexión con Certificado de la FNMT	10
7.7.3. Conexión con Certificado de la Asignatura	12
7.7.4. Verificación de Seguridad en el Navegador	13
7.8. Conclusión	14

Índice de figuras

7.1. Extracción de los componentes criptográficos del certificado de la FNMT. .	4
7.2. Creación y traslado del anillo de confianza <code>cas_confiables.crt</code> al directorio del sistema.	4
7.3. Habilitación del módulo CGI en el servidor Apache.	5
7.4. Estado de la configuración inicial del VirtualHost SSL.	5
7.5. Modificación del VirtualHost para habilitar mTLS y ejecución CGI.	5
7.6. Código fuente del <i>script</i> CGI <code>index.sh</code> leyendo variables SSL.	6
7.7. Modificación de permisos y propietario del archivo ejecutable.	6
7.8. Estado del servicio Apache tras la reconfiguración de autenticación mTLS y revisión de logs.	7
7.9. Trazas de conexión: Inicio de la negociación y envío de certificados.	8
7.10. Trazas de conexión: Exposición del certificado del servidor.	8
7.11. Trazas de conexión: Cifrado negociado y lista de autoridades aceptables. .	9
7.12. Trazas de conexión: Verificación exitosa (código 0) y cierre del Handshake.	9
7.13. Ajustes de privacidad y acceso al gestor de certificados en Firefox.	10
7.14. Verificación de los certificados personales (FNMT y UPLGC) importados en el navegador.	10
7.15. El navegador intercepta la conexión y solicita la identificación por certificado.	11
7.16. El <i>script</i> CGI expone los datos capturados. Se observa la CA Emisora: AC FNMT Usuarios.	11
7.17. Detalles de la conexión segura establecida desde la barra de estado del navegador.	12
7.18. Petición de identificación en Modo Privado, seleccionando el certificado de la asignatura.	12
7.19. El servidor acepta y procesa el certificado local (CA: Asignatura Seguridad de la Información).	13
7.20. Información técnica de seguridad: Conexión cifrada mediante TLS_AES_128_GCM.	13
7.21. Jerarquía de autoridades y propiedades del certificado validado por Firefox.	14

Capítulo 7

Servidor Web Seguro con Certificados de Cliente

En esta práctica número 7 se aborda la configuración avanzada del servidor web Apache seguro, implementando el mecanismo de Autenticación Mutua TLS (mTLS). El objetivo principal es configurar el servidor para que exija y valide certificados de cliente emitidos por múltiples Autoridades de Certificación (CA), específicamente la CA de la asignatura y una autoridad oficial de la Fábrica Nacional de Moneda y Timbre (FNMT).

Asimismo, se sustituirá el documento estático de prueba por un *script* CGI capaz de interpretar y mostrar dinámicamente las variables de entorno proporcionadas por la sesión SSL.

Para cumplir con los requisitos de trazabilidad y autoría, el “prompt” de la consola ha sido personalizado previamente con el nombre del autor:

```
export PS1="Sergio Acosta > "
```

7.1. Preparación de los Certificados (Asignatura y FNMT)

Como se dispone de un certificado oficial emitido por la FNMT en formato PKCS#12 (.p12), se utilizará este como la autoridad adicional requerida para la práctica, demostrando la integración de autoridades comerciales en un entorno local.

Para que Apache y las herramientas de consola (`openssl`) puedan trabajar con estas credenciales, es necesario desglosar el archivo .p12 en sus componentes fundamentales: el certificado del cliente, su clave privada y el certificado de la autoridad emisora. Se ejecutan los siguientes comandos para la extracción, utilizando la directiva `-nodes` para la clave privada, lo que suprime el cifrado local y agiliza las pruebas de conexión en consola:

```
openssl pkcs12 -in ACOSTA_QUINTANA_SERGIO___43836897R.p12 \
    -clcerts -nokeys -out cliente_fnmt.crt
openssl pkcs12 -in ACOSTA_QUINTANA_SERGIO___43836897R.p12 \
    -nocerts -out cliente_fnmt.key -nodes
openssl pkcs12 -in ACOSTA_QUINTANA_SERGIO___43836897R.p12 \
```

```
-cacerts -nokeys -out ca_fnmt.crt
```

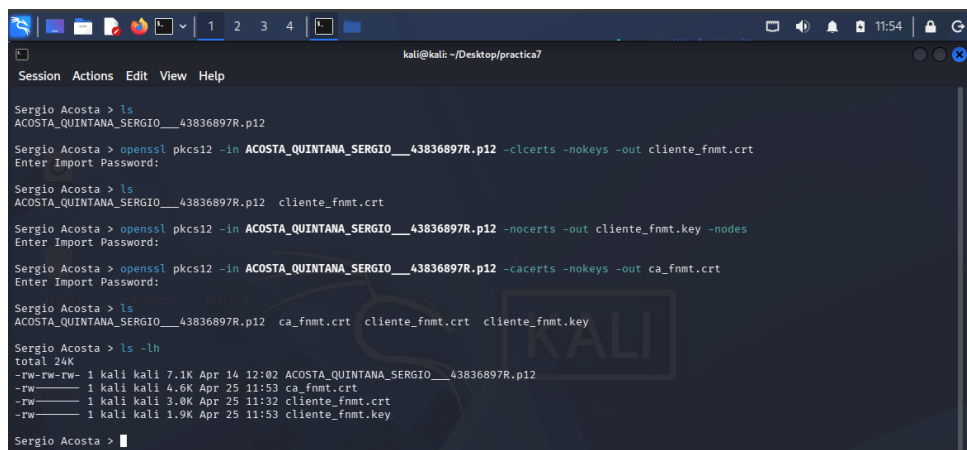


Figura 7.1: Extracción de los componentes criptográficos del certificado de la FNMT.

7.2. Creación del Fichero de CAs Confiables para Apache

El servidor Apache requiere un único archivo de directivas que consolide todas las Autoridades de Certificación en las que debe confiar. Se juntan el certificado raíz de la asignatura (Práctica 6) con el certificado raíz de la FNMT recién extraído.

```
cp ../practica6/ca.crt ./ca_asignatura.crt
cat ca_asignatura.crt ca_fnmt.crt > cas_confiables.crt
sudo cp cas_confiables.crt /etc/ssl/certs/
```

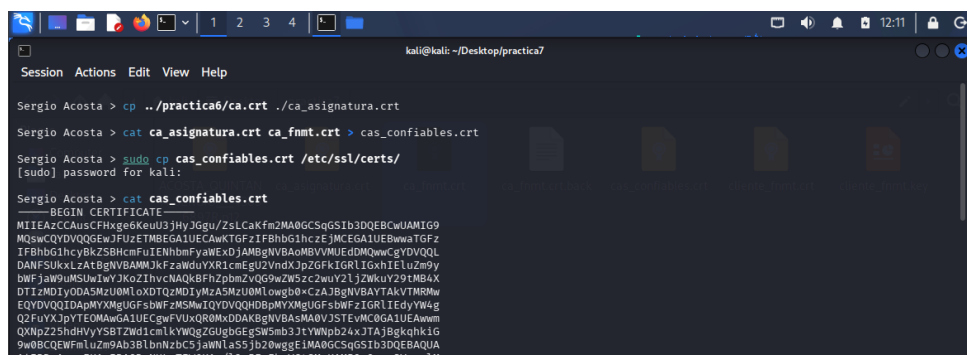
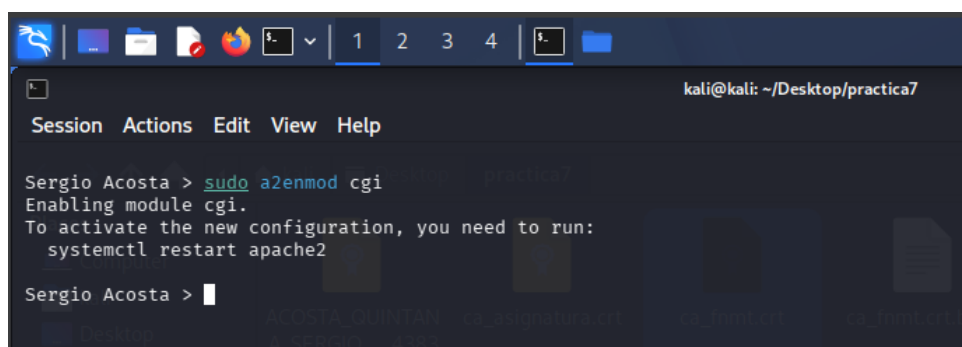


Figura 7.2: Creación y traslado del anillo de confianza `cas_confiables.crt` al directorio del sistema.

7.3. Modificación de la Configuración de Apache

Para permitir que el servidor pueda ejecutar *scripts* dinámicos y leer datos de la conexión TLS, se habilita en primer lugar el módulo CGI del servidor web:

```
sudo a2enmod cgi
```



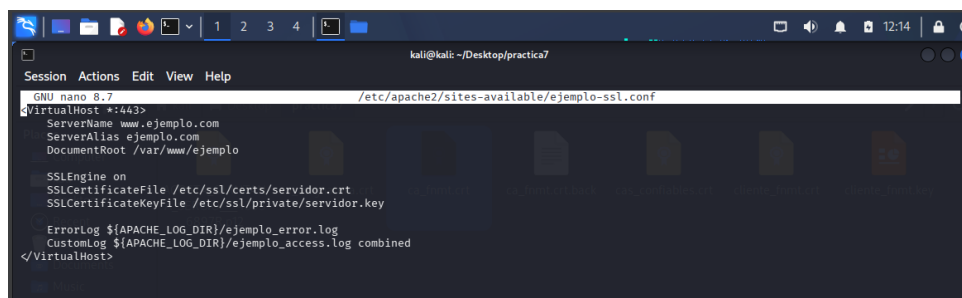
```
kali@kali: ~/Desktop/practica7
Session Actions Edit View Help

Sergio Acosta > sudo a2enmod cgi
Enabling module cgi.
To activate the new configuration, you need to run:
    systemctl restart apache2

Sergio Acosta > 
```

Figura 7.3: Habilitación del módulo CGI en el servidor Apache.

Posteriormente, se edita la configuración del *VirtualHost* (`ejemplo-ssl.conf`). Se añaden las directivas `SSLVerifyClient require` para hacer obligatoria la presentación del certificado, y se configura el directorio raíz para permitir la ejecución CGI e inyectar los metadatos del certificado del cliente en las variables de entorno mediante `SSLOptions +StdEnvVars`.

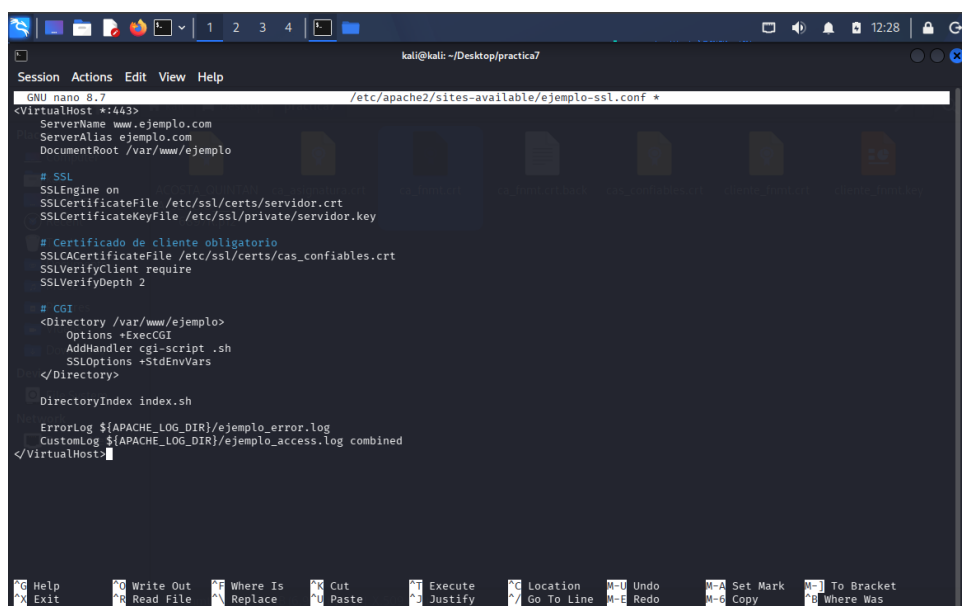


```
kali@kali: ~/Desktop/practica7
GNU nano 8.7 /etc/apache2/sites-available/ejemplo-ssl.conf
VirtualHost *:443>
ServerName www.ejemplo.com
ServerAlias ejemplo.com
DocumentRoot /var/www/ejemplo

SSLEngine on
SSLCertificateFile /etc/ssl/certs/servidor.crt
SSLCertificateKeyFile /etc/ssl/private/servidor.key

ErrorLog ${APACHE_LOG_DIR}/ejemplo_error.log
CustomLog ${APACHE_LOG_DIR}/ejemplo_access.log combined
</VirtualHost>
```

Figura 7.4: Estado de la configuración inicial del VirtualHost SSL.



```
kali@kali: ~/Desktop/practica7
GNU nano 8.7 /etc/apache2/sites-available/ejemplo-ssl.conf *
VirtualHost *:443>
ServerName www.ejemplo.com
ServerAlias ejemplo.com
DocumentRoot /var/www/ejemplo

# SSL
SSLEngine on
SSLCertificateFile /etc/ssl/certs/servidor.crt
SSLCertificateKeyFile /etc/ssl/private/servidor.key

# Certificado de cliente obligatorio
SSLCACertificateFile /etc/ssl/certs/cas_confiabiles.crt
SSLVerifyClient require
SSLVerifyDepth 2

# CGI
<Directory /var/www/ejemplo>
    Options +ExecCGI
    AddHandler cgi-script .sh
    SSLOptions +StdEnvVars
</Directory>

DirectoryIndex index.sh

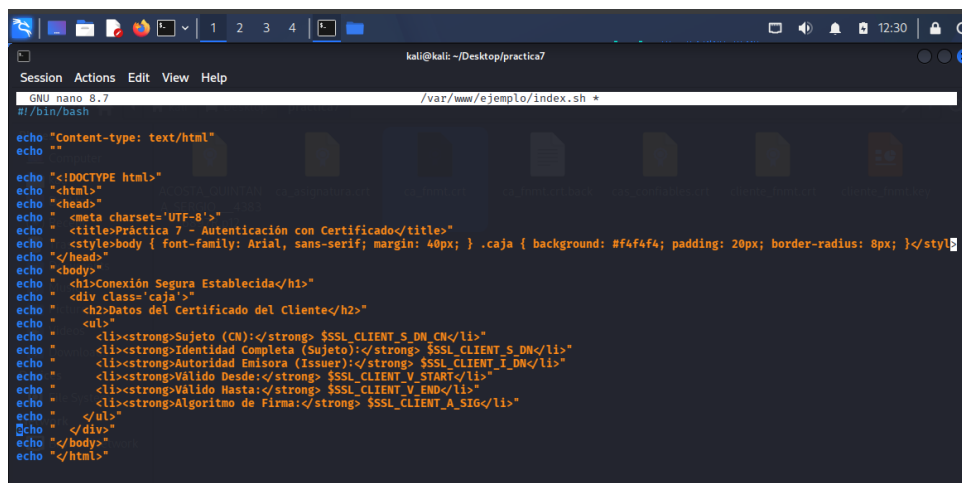
ErrorLog ${APACHE_LOG_DIR}/ejemplo_error.log
CustomLog ${APACHE_LOG_DIR}/ejemplo_access.log combined
</VirtualHost>
```

Figura 7.5: Modificación del VirtualHost para habilitar mTLS y ejecución CGI.

7.4. Instalación del Script CGI

Se reemplaza el archivo estático `index.html` por un *script* escrito en Bash (`index.sh`). Este *script* lee las variables de entorno que Apache genera a partir del certificado del cliente (sin recurrir a comandos locales de `openssl`) y renderizarlas en código HTML estructurado.

```
sudo mv /var/www/ejemplo/index.html /var/www/ejemplo/index.html.bak
sudo nano /var/www/ejemplo/index.sh
```



```
GNU nano 8.7 /var/www/ejemplo/index.sh *
#!/bin/bash

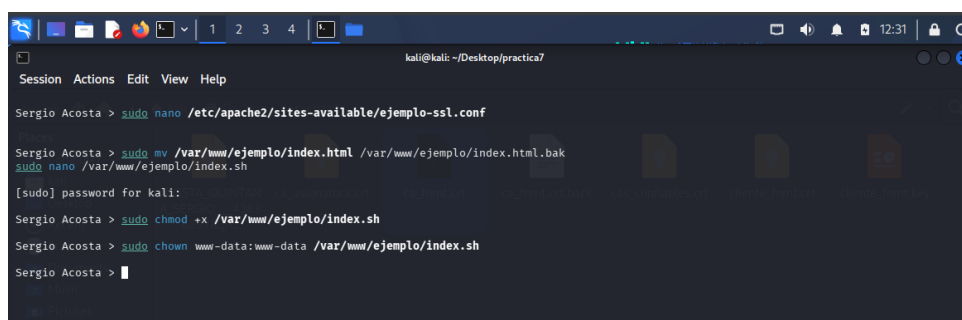
echo "Content-type: text/html"
echo ""

echo "<!DOCTYPE html>"
echo "<html>"
echo "<head>"
echo "<meta charset='UTF-8'>"
echo "<title>Práctica 7 - Autenticación con Certificado</title>"
echo "<style>body { font-family: Arial, sans-serif; margin: 40px; } .caja { background: #f4f4f4; padding: 20px; border-radius: 8px; }</style>"
echo "</head>"
echo "<body>"
echo "<h1>Conexión Segura Establecida</h1>"
echo "<div class='caja'>"
echo "<h2>Datos del Certificado del Cliente</h2>"
echo "<ul>"
echo "<li><strong>Sujeto (CN):</strong> $SSL_CLIENT_S_DN_CN</li>"
echo "<li><strong>Identidad Completa (Sujeto):</strong> $SSL_CLIENT_S_DN</li>"
echo "<li><strong>Autoridad Emisora (Issuer):</strong> $SSL_CLIENT_I_DN</li>"
echo "<li><strong>Válido Desde:</strong> $SSL_CLIENT_V_START</li>"
echo "<li><strong>Válido Hasta:</strong> $SSL_CLIENT_V_END</li>"
echo "<li><strong>Algoritmo de Firma:</strong> $SSL_CLIENT_A_SIG</li>"
echo "</ul>"
echo "</div>"
echo "</body>"
echo "</html>"
```

Figura 7.6: Código fuente del *script* CGI `index.sh` leyendo variables SSL.

Para que el servidor web pueda invocar el *script*, se otorgan los permisos de ejecución pertinentes y se establece a `www-data` como propietario del archivo:

```
sudo chmod +x /var/www/ejemplo/index.sh
sudo chown www-data:www-data /var/www/ejemplo/index.sh
```



```
kali@kali: ~/Desktop/practica7

Sergio Acosta > sudo nano /etc/apache2/sites-available/ejemplo-ssl.conf

Sergio Acosta > sudo mv /var/www/ejemplo/index.html /var/www/ejemplo/index.html.bak
sudo nano /var/www/ejemplo/index.sh

[sudo] password for kali:

Sergio Acosta > sudo chmod +x /var/www/ejemplo/index.sh

Sergio Acosta > sudo chown www-data:www-data /var/www/ejemplo/index.sh

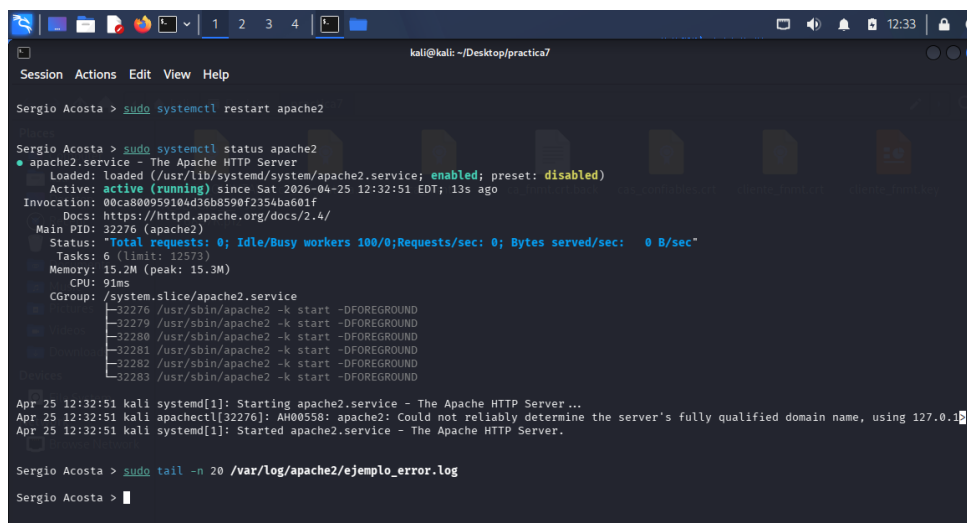
Sergio Acosta >
```

Figura 7.7: Modificación de permisos y propietario del archivo ejecutable.

7.5. Rearranque de Apache y Verificación

Para aplicar todas las modificaciones a nivel de arquitectura, se reinicia el servicio de Apache y se verifica su estado para confirmar que no existen fallos sintácticos en el *VirtualHost* que impidan su carga.

```
sudo systemctl restart apache2
sudo systemctl status apache2
sudo tail -n 20 /var/logs/apache2/ejemplo_error.log
```



```
kali@kali: ~/Desktop/practica7
Sergio Acosta > sudo systemctl restart apache2

Sergio Acosta > sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-04-25 12:32:51 EDT; 13s ago
     Invocation: 00ca80959104d36b8590f2354ba601f
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 32276 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 6 (limit: 12573)
       Memory: 15.2M (peak: 15.3M)
          CPU: 91ms
    CGroup: /system.slice/apache2.service
            └─32276 /usr/sbin/apache2 -k start -DFOREGROUND
              └─32279 /usr/sbin/apache2 -k start -DFOREGROUND
                └─32280 /usr/sbin/apache2 -k start -DFOREGROUND
                  └─32281 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─32282 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─32283 /usr/sbin/apache2 -k start -DFOREGROUND

Apr 25 12:32:51 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Apr 25 12:32:51 kali apache2[32276]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1
Apr 25 12:32:51 kali systemd[1]: Started apache2.service - The Apache HTTP Server.

Sergio Acosta > sudo tail -n 20 /var/log/apache2/ejemplo_error.log
Sergio Acosta >
```

Figura 7.8: Estado del servicio Apache tras la reconfiguración de autenticación mTLS y revisión de logs.

7.6. Conexión de prueba con openssl s_client

Antes de probarlo en el navegador, se realiza una prueba donde el cliente de terminal interactúa con el puerto 443 adjuntando explícitamente el certificado y la clave privada de la FNMT frente a nuestra lista de CAs confiables.

```
openssl s_client -connect www.ejemplo.com:443 -cert cliente_fnmt.crt \
-key cliente_fnmt.key -CAfile cas_confiables.crt -servername www.ejemplo.com
```

Las siguientes capturas secuenciales demuestran que la negociación del *Handshake* se completa exitosamente, culminando con el código `Verify return code: 0 (ok)` y estableciendo una sesión TLS 1.3 robusta sin emitir alertas de interrupción por CA desconocida.


```

kali@kali: ~/Desktop/practica7
Session Actions Edit View Help

Sergio Acosta > openssl s_client -connect www.ejemplo.com:443 -cert cliente_fmmt.crt -key cliente_fmmt.key -CAfile cas_confiables.crt -servername
ame www.ejemplo.com -showcerts
Connecting to 127.0.0.1
CONNECTED(00000003)
depth=1 C=ES, ST=Las Palmas, L=Las Palmas de Gran Canaria, O=ULPGC, OU=EII, CN=Asignatura Seguridad de la Informacion, emailAddress=info@opensa
sl.cicei.com
verify return:1
depth=0 C=ES, ST=Spain, L=Las Palmas, O=ULPGC, OU=EII, CN=www.ejemplo.com, emailAddress=sergioacostaquintana@gmail.com
verify return:1

Certificate chain
 0 s:C=ES, ST=Spain, L=Las Palmas, O=ULPGC, OU=EII, CN=www.ejemplo.com, emailAddress=sergioacostaquintana@gmail.com
 1 s:C=ES, ST=Las Palmas, L=Las Palmas de Gran Canaria, O=ULPGC, OU=EII, CN=Asignatura Seguridad de la Informacion, emailAddress=info@opensa
sl.cicei.com
a:KEY: RSA, 2048 (bit); sigalg: sha256WithRSAEncryption
v:NotBefore: Apr 12 16:16:10 2026 GMT; NotAfter: Apr 12 16:16:10 2027 GMT
-----BEGIN CERTIFICATE-----
MIIFNjCCBBGwIBAgIUZ2ZWwE9/hY7B9DJgrRxtwK0s8sWQYJKoZIhvcNAQEL
BQAwgboxCzAJBgNVBAYTAkVTMRMwEQYDVQQIDApMYXMGUFGSbWZFMScwI
DBpMYXMGUFGSbWZFIgRlIEdyYW4gQ2FuYXpYpYTEOMAwGA1UECgwFVUx
R0MxMDAKBgNVBAsMA0VJSTEVMC0GA1UEAwMwQXpZ25hdHVVYySBT2Wd1cm
kYwQZGUGbGEGSbW5mb3JlYWNpb24xJTAJBGkq
-----END CERTIFICATE-----

```

Figura 7.9: Trazas de conexión: Inicio de la negociación y envío de certificados.

```

kali@kali: ~/Desktop/practica7
Session Actions Edit View Help

LDmeyJAWoGCl+dStMm0hjaCzI3j7Z65oPyZ6LUZ2GQw2E/DfWdCH+nTD0Lr+SB
28G3qMauBDrOfVl0eLkjdMzLbLkX7y6w+hsHvL3mVfDKLJNjSxYXf+i0IYaxoRe
GutIovKbNEloVCDfYulwscZ0McjHCHUHK4jaqzXULapDOOHtm2AMxR3Wd/12Qe5x
SUnb2ozuDM/JbK2e7fji2ryEdjYtZJ3Gd2pW9y0+J4Vf4lFkLULANf
-----END CERTIFICATE-----
1 s:C=ES, ST=Las Palmas, L=Las Palmas de Gran Canaria, O=ULPGC, OU=EII, CN=Asignatura Seguridad de la Informacion, emailAddress=info@opensa
sl.cicei.com
1 s:C=ES, ST=Las Palmas, L=Las Palmas de Gran Canaria, O=ULPGC, OU=EII, CN=Asignatura Seguridad de la Informacion, emailAddress=info@opensa
sl.cicei.com
a:KEY: RSA, 2048 (bit); sigalg: sha256WithRSAEncryption
v:NotBefore: Feb 28 09:35:42 2023 GMT; NotAfter: Feb 23 09:35:42 2043 GMT
-----BEGIN CERTIFICATE-----
MIIEA2CAUSCFHxge6KeuU3jHyJGgu/ZsLcAKfm2MA0GCSqGSIb3DQEBCwUAMIG9
MQswCQYDVQQGEwJFuzETMBEGA1UECAwKTGFzIFBhbG1hc2EjMCEGA1UEBmwaTGfz
IFBhbG1hc2ZSBHcmFuIEhnbmFyaWExDjAMBGNVBAoMBVJMUEDDMQwCgYDVQQL
DANFSUxR0MxMDAKBgNVBAsMA0VJSTEVMC0GA1UEAwMwQXpZ25hdHVVYySBT2Wd1cm
kYwQZGUGbGEGSbW5mb3JlYWNpb24xJTAJBGkq
-----END CERTIFICATE-----
Server certificate
subject=C=ES, ST=Spain, L=Las Palmas, O=ULPGC, OU=EII, CN=www.ejemplo.com, emailAddress=sergioacostaquintana@gmail.com
issuer=C=ES, ST=Las Palmas, L=Las Palmas de Gran Canaria, O=ULPGC, OU=EII, CN=Asignatura Seguridad de la Informacion, emailAddress=info@opensa
sl.cicei.com
-----END CERTIFICATE-----

```

Figura 7.10: Trazas de conexión: Exposición del certificado del servidor.

```

kali@kali: ~/Desktop/practica7

Session Actions Edit View Help

subject=C=ES, ST=Spain, L=Las Palmas, O=ULPGC, OU=EII, CN=www.ejemplo.com, emailAddress=sergioacostaquintana@gmail.com
issuer=C=ES, ST=Las Palmas, L=Las Palmas de Gran Canaria, O=ULPGC, OU=EII, CN=Asignatura Seguridad de la Informacion, emailAddress=info@openssl.cicei.com

Acceptable client certificate CA names
C=ES, ST=Las Palmas, L=Las Palmas de Gran Canaria, O=ULPGC, OU=EII, CN=Asignatura Seguridad de la Informacion, emailAddress=info@openssl.cicei.com
C=ES, O=FNMT-RCM, OU=Ceres, CN=AC FNMT Usuarios
C=ES, O=FNMT-RCM, OU=AC RAIZ FNMT-RCM
Requested Signature Algorithms: id-mt-dsa-65:id-mt-dsa-87:id-mt-dsa-44:ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:ed25519:ed448:ecdsa_brainpoolP256r1_sha256:ecdsa_brainpoolP384r1_sha384:ecdsa_brainpoolP512r1_sha512:rsa_pss_sha256:rsa_pss_sha384:rsa_pss_sha512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512
Shared Requested Signature Algorithms: id-mt-dsa-65:id-mt-dsa-87:id-mt-dsa-44:ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:ed25519:ed448:ecdsa_brainpoolP256r1_sha256:ecdsa_brainpoolP384r1_sha384:ecdsa_brainpoolP512r1_sha512:rsa_pss_sha256:rsa_pss_sha384:rsa_pss_sha512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512
Peer signing digest: SHA256
Peer signature type: rsa_pss_rsae_sha256
Negotiated TLS1.3 group: X25519MLKEM768

SSL handshake has read 4448 bytes and written 5880 bytes
Verification: OK

New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Protocol: TLSv1.3
Server public key is 2048 bit
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)

Post-Handshake New Session Ticket arrived:
SSL-Session:
    Protocol : TLSv1.3
    Cipher : TLS_AES_256_GCM_SHA384
    Session-ID: 10F29312E7C5A32324143B175B48B02A6FAEBE0ED4A8907227A4B0146B47BFD0C
    Session-ID-ctx:

```

Figura 7.11: Trazas de conexión: Cifrado negociado y lista de autoridades aceptables.

```

kali@kali: ~/Desktop/practica7

Session Actions Edit View Help

SRP username: None
TLS session ticket lifetime hint: 300 (seconds)
TLS session ticket:
0000 - 33 28 36 9e cc 16 97 16-f6 28 0a a3 22 e5 f3 12 3(6.....{..}...
0010 - 48 5c 05 13 83 1b 0b 35-5f 67 fa 13 12 9f eb 1b H\.....5.g.....

Start Time: 1777135265
Timeout : 7200 (sec)
Verify return code: 0 (ok)
Extended master secret: no
Max Early Data: 0

read R BLOCK

Post-Handshake New Session Ticket arrived:
SSL-Session:
    Protocol : TLSv1.3
    Cipher : TLS_AES_256_GCM_SHA384
    Session-ID: 646ED68FEECAD3C7E4EE14441E3B0CB13EDB05707032156E8E1F61EC6BF7714F
    Session-ID-ctx:
    Resumption PSK: F7F84F20467AEA34E29A5E099846AE73314AF3D4EBABFBBAD1EC3C2CCBFEE753B3D690F274F2057A6F2B6B88558A09D
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    TLS session ticket lifetime hint: 300 (seconds)
    TLS session ticket:
0000 - 81 a4 b7 af 3d e5 57 50-f4 a8 5a 2e 31 8c 79 36 ....=.WP..Z.1.y6
0010 - 5b 40 65 22 18 de 14 0e-1e 1b 60 2f 58 9b 63 83 [0e"....."/X.c.

Start Time: 1777135265
Timeout : 7200 (sec)
Verify return code: 0 (ok)
Extended master secret: no
Max Early Data: 0

read R BLOCK
closed

Sergio Acosta > ss

```

Figura 7.12: Trazas de conexión: Verificación exitosa (código 0) y cierre del Handshake.

7.7. Pruebas Finales en Firefox

7.7.1. Importación de Certificados

Se comprueba que el gestor interno de certificados de Mozilla Firefox contiene correctamente importados tanto el certificado personal de la asignatura (Práctica 3) como el certificado oficial emitido por la FNMT.

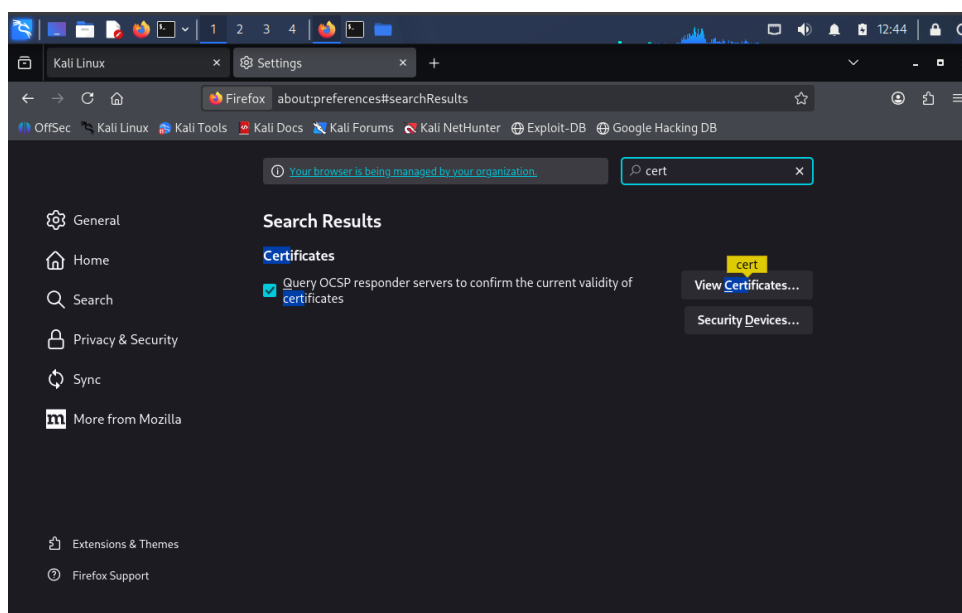


Figura 7.13: Ajustes de privacidad y acceso al gestor de certificados en Firefox.

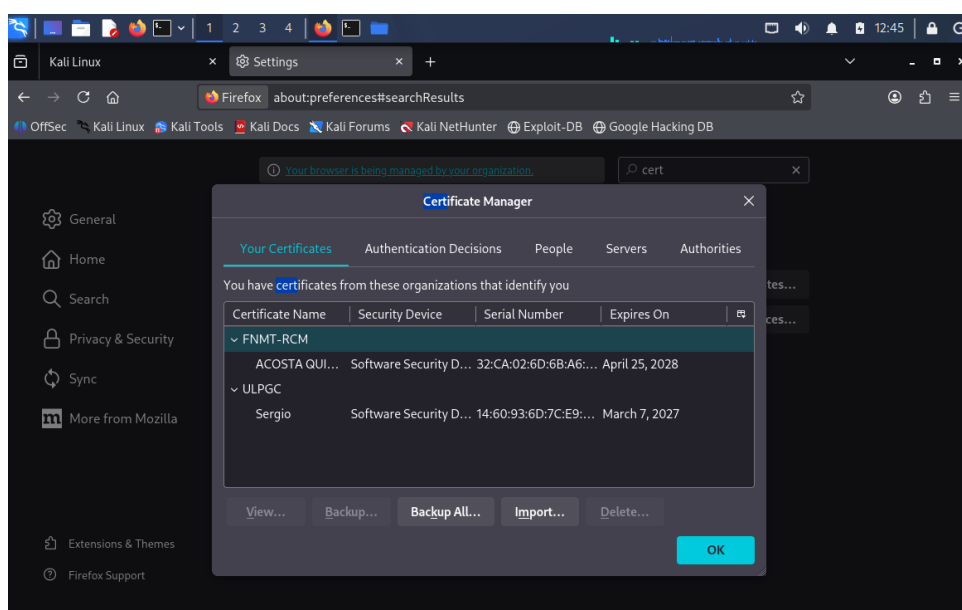


Figura 7.14: Verificación de los certificados personales (FNMT y ULPGC) importados en el navegador.

7.7.2. Conexión con Certificado de la FNMT

Al acceder al servidor de manera convencional, el navegador detecta la exigencia `SSLVerifyClient require` de Apache y despliega un menú solicitando la identidad del usuario. Se selecciona el certificado de la FNMT.

El *script* Bash captura con éxito las variables de entorno correspondientes a la autoridad externa y las plasma en el documento HTML renderizado.

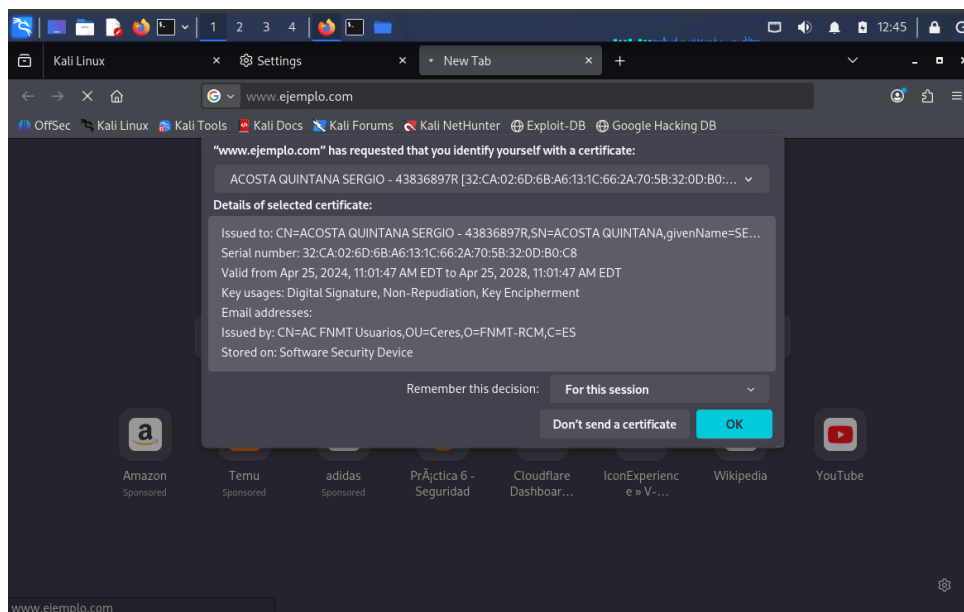
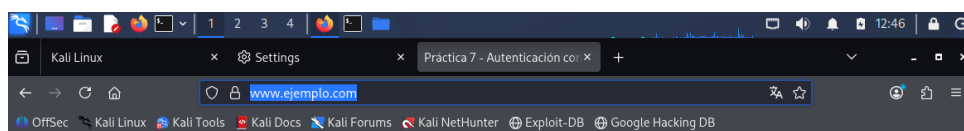


Figura 7.15: El navegador intercepta la conexión y solicita la identificación por certificado.



Conexión Segura Establecida

Datos del Certificado del Cliente

- **Sujeto (CN):** ACOSTA QUINTANA SERGIO - 43836897R
- **Identidad Completa (Sujeto):** CN=ACOSTA QUINTANA SERGIO - 43836897R, SN=ACOSTA QUINTANA, GN=SERGIO, serialNumber=IDCES-43836897R, C=ES
- **Autoridad Emisora (Issuer):** CN=AC FNMT Usuarios, OU=Ceres, O=FNMT-RCM, C=ES
- **Válido Desde:** Apr 25 15:01:47 2024 GMT
- **Válido Hasta:** Apr 25 15:01:47 2028 GMT
- **Algoritmo de Firma:** sha256WithRSAEncryption

Figura 7.16: El *script* CGI expone los datos capturados. Se observa la CA Emisora: AC FNMT Usuarios.

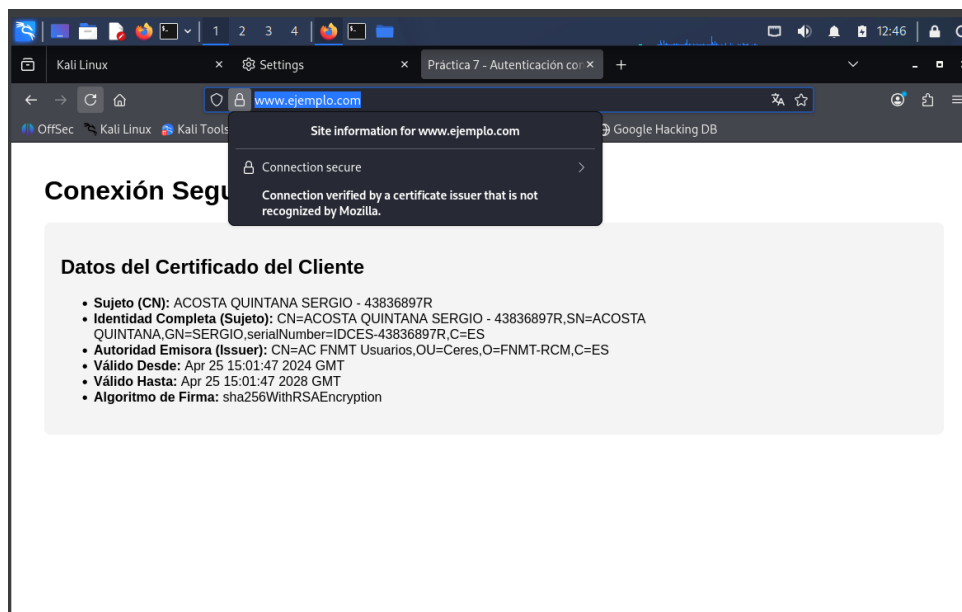


Figura 7.17: Detalles de la conexión segura establecida desde la barra de estado del navegador.

7.7.3. Conexión con Certificado de la Asignatura

Para evitar conflictos de caché SSL en el navegador, se realiza una segunda prueba abriendo una sesión en modo Privado/Incógnito. En este caso se selecciona el certificado emitido por la Autoridad Docente.

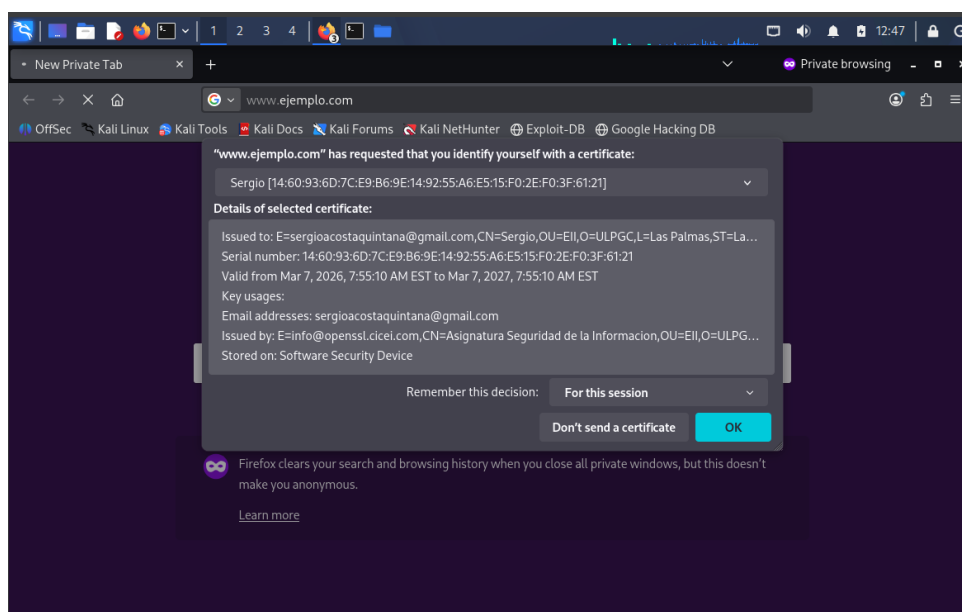


Figura 7.18: Petición de identificación en Modo Privado, seleccionando el certificado de la asignatura.

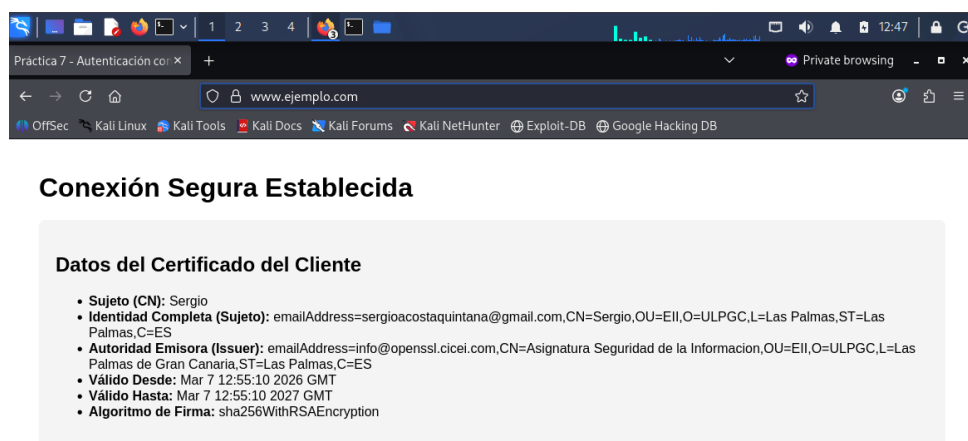


Figura 7.19: El servidor acepta y procesa el certificado local (CA: Asignatura Seguridad de la Información).

7.7.4. Verificación de Seguridad en el Navegador

Por último, se audita la información técnica de la conexión para certificar el cifrado en curso y la lectura correcta de la cadena de confianza por parte de Firefox.

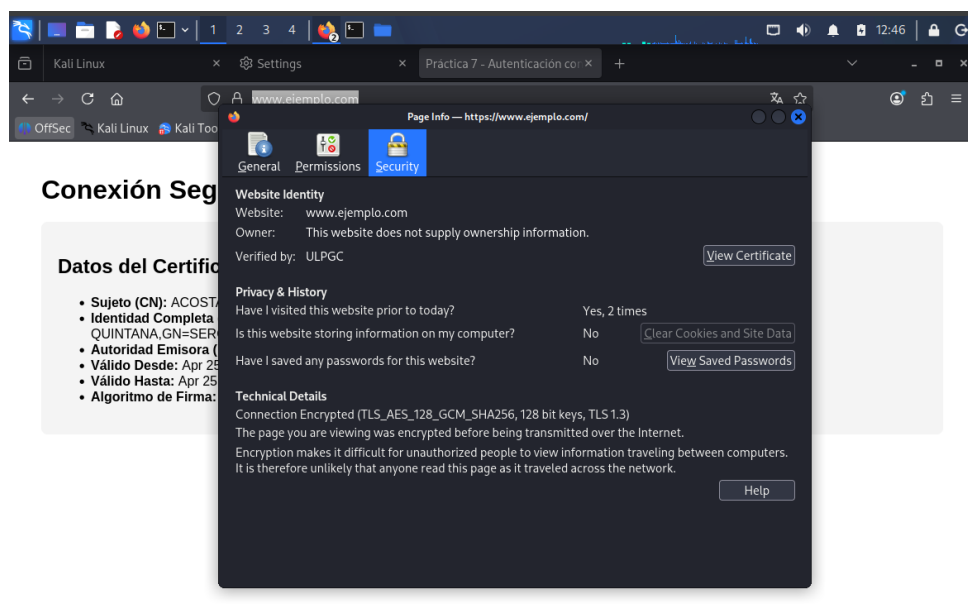


Figura 7.20: Información técnica de seguridad: Conexión cifrada mediante TLS_AES_128_GCM.

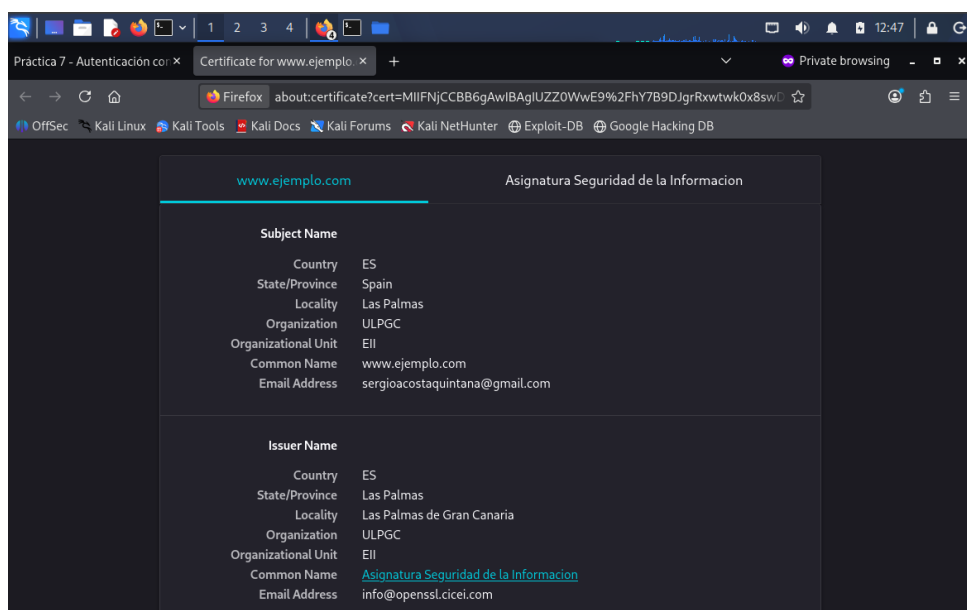


Figura 7.21: Jerarquía de autoridades y propiedades del certificado validado por Firefox.

7.8. Conclusión

En esta práctica se ha configurado un servidor web seguro utilizando autenticación mutua TLS (mTLS), lo que permite que tanto el servidor como el cliente se identifiquen mediante certificados digitales. Se ha conseguido que Apache acepte certificados de dos autoridades distintas (la de la asignatura y la FNMT), comprobando que ambas conexiones funcionan correctamente.

Además, se ha implementado un script CGI que muestra la información del certificado del cliente usando las variables de entorno proporcionadas por Apache, sin necesidad de usar herramientas externas como `openssl`.

Las pruebas realizadas, tanto con `openssl` como con el navegador Firefox, han permitido verificar que la configuración es correcta y que la conexión se establece de forma segura. En general, esta práctica ha ayudado a entender mejor cómo funciona la autenticación con certificados y cómo se aplica en un entorno real.

Bibliografía

- [1] The OpenSSL Project, “OpenSSL Manpages y Documentación Oficial,” [En línea]. Disponible: <https://www.openssl.org/docs/>. [Accedido: 3 de mayo de 2026].
- [2] The Apache Software Foundation, “Apache HTTP Server Documentation,” [En línea]. Disponible: <https://httpd.apache.org/docs/>. [Accedido: 3 de mayo de 2026].
- [3] D. Cooper et al., “Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile,” IETF RFC 5280, May 2008.
- [4] Google, “Gemini: Modelos de IA generativa,” [En línea]. Herramienta utilizada para el contraste de información teórica sobre mTLS e infraestructuras PKI. Disponible: <https://gemini.google.com/>. [Accedido: 3 de mayo de 2026].
- [5] OpenAI, “ChatGPT: Asistencia en redacción técnica y estructuración,” [En línea]. Apoyo en la cohesión textual y revisión del formato del documento en LaTeX. Disponible: <https://chat.openai.com/>. [Accedido: 3 de mayo de 2026].